

湖北汽车工业学院
HUBEI UNIVERSITY OF AUTOMOTIVE TECHNOLOGY

智能网联汽车学院

移动终端软件开发

实验指导书

制定人：张　毅

审核人：张吴波

日　期：2026年8月

目 录

目 录.....1
实验三 线程与网络通信.....2
一、实验内容.....2
程序 1 TCP 流套接字通信.....2
程序 2 HTTP 网络请求.....3
程序 3 WebView 加载网页.....4
程序 4 LocationManager 定位.....5
二、思考题.....6
2.1 习题.....6
2.2 实验小结.....8

实验三　线程与网络通信

一、实验内容

程序 1 TCP 流套接字通信

（1）写出 Java 服务器端和 Android 客户端建立连接、发送与接收消息的关键代码，标明服务器 IP、端口、网络权限及子线程处理。

服务器 IP：模拟器中使用 10.0.2.2（指向宿主机），真机中填写主机局域网 IP（如 10.28.137.252）；端口：8888；网络权限：android.permission.INTERNET；客户端网络操作全部放在子线程中执行。

```
// ========== 服务器端 TcpServer.java（Java Project）==========
ServerSocket serverSocket = new ServerSocket(8888);       // 端口 8888（>1024）
System.out.println("等待接收客户端的连接: ");
Socket socket = serverSocket.accept();                    // 等待客户端连接
System.out.println("建立连接成功");
// 读取客户端发来的信息
BufferedReader in = new BufferedReader(new InputStreamReader(
       socket.getInputStream(), "UTF-8"));
String str = in.readLine();
System.out.println("客户端发来的信息:" + str);
// 向客户端发送消息
PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(
       socket.getOutputStream(), "UTF-8")), true);
out.println("hello, I am Server!");
in.close(); out.close(); socket.close();


// ========== Android 客户端 TcpClientActivity（子线程）==========
// AndroidManifest.xml 中声明网络权限：
//   <uses-permission android:name="android.permission.INTERNET"/>
new Thread(new Runnable() {               // 网络操作放在子线程，避免阻塞主线程
    public void run() {
        try {
            // 服务器 IP 和端口：模拟器用 10.0.2.2 访问主机
            Socket socket = new Socket("10.0.2.2", 8888);
            // 通过 PrintWriter 向服务器发送信息
            PrintWriter out = new PrintWriter(new BufferedWriter(new
                    OutputStreamWriter(socket.getOutputStream(), "UTF-8")), true);
            out.println("I am client!");
            // 通过 BufferedReader 读取服务器返回的信息
            BufferedReader br = new BufferedReader(new InputStreamReader(
                    socket.getInputStream(), "UTF-8"));
            final String sMessage = br.readLine();
            out.close(); br.close(); socket.close();
            // 回到主线程，把双方收发的信息显示在 EditText 中
            runOnUiThread(new Runnable() { public void run() {
                clientText.setText("收到服务器消息：" + sMessage);
            }});
        } catch (Exception e) { e.printStackTrace(); }
    }
}).start();
```

（2）粘贴服务器端与 Android 客户端通信的运行截图，简要标注双方发送和接收的内容。



图 1-1　服务器端运行结果：
收到客户端信息 I am client!，回复 hello, I am Server!



图 1-2　Android 客户端：
发送 I am client!，收到 hello, I am Server!

程序 2 HTTP 网络请求

（1）写出 GET、POST 请求及获取响应结果的关键代码，标明请求地址、参数和所用网络 API。

请求地址：http://10.0.2.2:8080/test.jsp；参数：name=wwh、pwd=123456；所用网络 API：java.net.HttpURLConnection（实验指导书中的 HttpClient 在新版本 Android 中已被废弃，因此改用 HttpURLConnection）。服务端使用 Java 编写的 HttpTestServer 模拟 test.jsp 的 GET/POST 处理逻辑。

```
// ========== GET 请求：参数拼在网址后面 ==========
String params = "name=" + URLEncoder.encode(name, "UTF-8")
       + "&pwd=" + URLEncoder.encode(pwd, "UTF-8");
URL url = new URL("http://10.0.2.2:8080/test.jsp?" + params);
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
conn.setConnectTimeout(8000); conn.setReadTimeout(8000);
conn.setRequestMethod("GET");
int code = conn.getResponseCode();          // 200 表示请求成功
if (code == HttpURLConnection.HTTP_OK) {
    BufferedReader br = new BufferedReader(
            new InputStreamReader(conn.getInputStream(), "UTF-8"));
    StringBuilder sb = new StringBuilder(); String line;
    while ((line = br.readLine()) != null) sb.append(line);
    br.close();
    // 返回结果：GET was used. Hello, wwh
    tvResult.setText("响应结果：" + sb.toString());
}
conn.disconnect();


// ========== POST 请求：参数放在请求体中 ==========
HttpURLConnection conn = (HttpURLConnection) new URL(
       "http://10.0.2.2:8080/test.jsp").openConnection();
conn.setRequestMethod("POST");
conn.setDoOutput(true);
conn.setRequestProperty("Content-Type",
       "application/x-www-form-urlencoded; charset=UTF-8");
OutputStream os = conn.getOutputStream();
os.write(("name=" + URLEncoder.encode(name, "UTF-8") + "&pwd="
       + URLEncoder.encode(pwd, "UTF-8")).getBytes("UTF-8"));
os.flush(); os.close();
int code = conn.getResponseCode();          // 读取状态码和响应流
```

（2）粘贴 GET、POST 请求及响应结果的运行截图，简要标注操作与结果。



图 2-1 GET 请求：参数在网址中，返回 GET was used. Hello, wwh



图 2-2 POST 请求：参数在请求体中，返回 POST was used. Hello, wwh

程序 3 WebView 加载网页

（1）写出 WebView 初始化、加载指定网址及页面加载处理的关键代码。

```
// 1. 初始化 WebView：支持 JavaScript、DOM 存储和缩放
WebSettings settings = webView.getSettings();
settings.setJavaScriptEnabled(true);
settings.setDomStorageEnabled(true);
settings.setBuiltInZoomControls(true);


// 2. 页面加载处理：通过 WebViewClient 监听加载过程
webView.setWebViewClient(new WebViewClient() {
    public void onPageStarted(WebView view, String url, Bitmap favicon) {
        tvStatus.setText("页面状态：正在加载 " + url);
    }
    public void onPageFinished(WebView view, String url) {
        tvStatus.setText("页面状态：加载完成 " + url);
    }
    public void onReceivedError(WebView view, int errorCode,
                                String description, String failingUrl) {
        tvStatus.setText("页面状态：加载失败（" + errorCode + "）" + description);
    }
});


// 3. 加载指定网址（按钮或回车触发）：网址缺少协议时自动补全 http://
String url = urlText.getText().toString().trim();
if (!url.startsWith("http://") && !url.startsWith("https://"))
    url = "http://" + url;
webView.loadUrl(url);       // 如 http://www.qq.com


// 4. AndroidManifest.xml：声明网络权限，并允许明文 HTTP 流量
// <uses-permission android:name="android.permission.INTERNET"/>
// <application android:usesCleartextTraffic="true" ... >
```

（2）粘贴输入网址和应用内网页加载结果的运行截图，标明访问的网址。



图 3-1　输入网址 http://www.qq.com
点击 GO，WebView 加载腾讯网首页



图 3-2　加载本地页面
http://10.0.2.2:8080/index.html

程序 4 LocationManager 定位

（1）写出定位权限申请、获取 LocationManager、接收定位结果及显示经纬度的关键代码。

```
<!-- 1. AndroidManifest.xml 中声明定位权限 -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>


// 2. 运行时动态申请定位权限（API 23 以后必须动态申请）
if (checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {
    requestPermissions(new String[]{Manifest.permission.ACCESS_FINE_LOCATION,
            Manifest.permission.ACCESS_COARSE_LOCATION}, REQUEST_LOCATION);
} else { locationManager_init(); }


// 3. 获取 LocationManager，注册定位监听
locationManager = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER,
        1000, 0, locationListener);       // 1 秒更新一次，最小距离 0 米


// 4. 实现 LocationListener，接收定位结果并显示经纬度
locationListener = new LocationListener() {
    public void onLocationChanged(Location location) {
        infoView.setText("定位时间：" + location.getTime()
                + "\n定位精度：" + location.getAccuracy() + " 米");
        locationView.setText("经度：" + location.getLongitude()
                + "\n纬度：" + location.getLatitude()
                + "\n海拔：" + location.getAltitude() + " 米");
    }
    public void onProviderEnabled(String provider)  { /* GPS 开启 */ }
    public void onProviderDisabled(String provider) { /* GPS 关闭 */ }
};
```

（2）粘贴获取并显示经纬度的运行截图，注明使用真机还是模拟器。



图 4-1 模拟器获取并显示经纬度（adb emu geo fix 设置：116.397, 39.909）

二、思考题

2.1 习题

1. TCP 通信中，Java 服务器端和 Android 客户端的编程步骤分别是什么？Android 客户端需要声明什么网络权限？

（1）Java 服务器端编程步骤：

① 创建 ServerSocket 对象并绑定监听端口（如 8888，端口号应大于 1024）；

② 调用 accept() 阻塞等待客户端连接，连接成功后得到 Socket 对象；

③ 通过 socket.getInputStream()/getOutputStream() 获取输入输出流，用 BufferedReader/PrintWriter 读写数据；

④ 通信结束后依次关闭输入流、输出流、Socket 和 ServerSocket。

（2）Android 客户端编程步骤：

① 在 AndroidManifest.xml 中声明网络权限；

② 新建子线程，在子线程中创建 Socket(服务器 IP, 端口) 与服务器建立连接（主线程不允许网络操作，否则抛出 NetworkOnMainThreadException）；

③ 用 PrintWriter 向服务器发送消息，用 BufferedReader 读取服务器返回的数据；

④ 通过 runOnUiThread() 或 Handler 回到主线程，把结果显示到界面；

⑤ 通信结束后关闭流和 Socket。

（3）Android 客户端需要声明 android.permission.INTERNET 网络权限；如需判断网络状态，还可以声明 android.permission.ACCESS_NETWORK_STATE。

2. 发送 HTTP GET 请求和 POST 请求的步骤分别是什么？

（1）GET 请求步骤：

① 用 URL 封装请求地址，参数以 ?name=…&pwd=… 的形式拼接在地址后面，并用 URLEncoder 对参数编码；

② 调用 openConnection() 得到 HttpURLConnection，setRequestMethod("GET") 并设置连接、读取超时；

③ 调用 getResponseCode() 获取响应状态码（200 表示成功）；

④ 通过 getInputStream() 获取输入流，用 BufferedReader 逐行读取响应内容；

⑤ 关闭输入流并 disconnect() 释放连接，把结果显示到界面。

（2）POST 请求步骤：

① 创建 HttpURLConnection 后 setRequestMethod("POST")，并设置 setDoOutput(true)；

② 设置请求头 Content-Type: application/x-www-form-urlencoded（UTF-8 编码）；

③ 把参数拼成 name=…&pwd=… 的字符串，写入 conn.getOutputStream() 请求体；

④ 其余步骤与 GET 相同：获取状态码、读取响应流、关闭连接。

（3）所有网络请求都要放在子线程中执行，避免阻塞主线程；本实验使用的网络 API 是 HttpURLConnection。

3. 怎样在 WebView 中加载一个网页？

① 在布局文件中添加 WebView 组件，在 Activity 中通过 findViewById() 获取 WebView 对象；

② 通过 webView.getSettings() 获取 WebSettings，调用 setJavaScriptEnabled(true) 支持 JavaScript，按需设置 setDomStorageEnabled(true)、缩放等属性；

③ 调用 webView.setWebViewClient(new WebViewClient(){…}) 设置页面加载监听，在 onPageStarted()、onPageFinished()、onReceivedError() 中分别处理加载开始、完成和失败；

④ 调用 webView.loadUrl("http://…") 加载指定网址，网址缺少协议时可自动补全 http://；

⑤ 在 AndroidManifest.xml 中声明 INTERNET 权限（加载 HTTP 网页还需允许明文流量）；重写 onBackPressed() 使返回键先回退网页历史，在 onDestroy() 中调用 webView.destroy() 释放资源。

4. 使用 LocationManager 进行定位的步骤是什么？

① 在 AndroidManifest.xml 中声明 ACCESS_FINE_LOCATION 和 ACCESS_COARSE_LOCATION 定位权限；

② 在运行时调用 requestPermissions() 动态申请定位权限，并在 onRequestPermissionsResult() 中处理授权结果（API 23 以后必须动态申请）；

③ 通过 getSystemService(Context.LOCATION_SERVICE) 获取 LocationManager 对象，用 isProviderEnabled() 判断 GPS/网络定位是否可用；

④ 实现 LocationListener 接口，在 onLocationChanged(Location location) 中获取经纬度、海拔、

精度和时间并显示到 TextView；

⑤ 调用 requestLocationUpdates(LocationManager.GPS_PROVIDER, 1000, 0, locationListener) 注册定位监听（参数依次为定位方式、最小更新时间、最小更新距离、监听器），退出时 removeUpdates() 移除监听；

⑥ 在模拟器中可通过 Extended Controls→Location 或 adb emu geo fix 设置经纬度，程序即可收到定位回调。

5. 附加题（选做）：编写 Android 应用程序加载百度地图，简述实现过程并粘贴运行截图。

附加题为选做内容。本次实验环境为离线环境，百度地图 SDK 需要申请 AK 并通过 Gradle 下载 SDK 依赖，无法完成 SDK 集成，因此未提交百度地图 SDK 的运行截图。其实现过程为：

① 在百度地图开放平台申请 AK，在 AndroidManifest.xml 中配置 <meta-data android:name="com.baidu.lbsapi.API_KEY" android:value="AK"/> 以及网络、定位权限；

② 在 build.gradle 中添加百度地图 SDK 依赖（BaiduMapSDK_Map、BaiduMapSDK_Location 等）；

③ 在布局中放置 MapView，在 Activity 中调用 SDKInitializer.initialize(getApplicationContext()) 初始化 SDK；

④ 通过 mapView.getMap() 获取 BaiduMap 对象，调用 setMyLocationEnabled(true) 显示定位图层，用 LocationClient 获取当前位置并 moveCamera 到该坐标。

2.2 实验小结

1 编程遇到的问题及解决方案（编程中遇到的问题以及程序调试过程等，分条写）

1. 现象：客户端报 NetworkOnMainThreadException，程序闪退。原因：在 onCreate() 主线程中直接创建 Socket。解决：把建立连接、收发数据全部放进 new Thread(){…}.start() 子线程，并用 runOnUiThread() 回到主线程更新 EditText。

2. 现象：客户端连接失败，提示 Connection refused。原因：服务器没有先运行，或客户端 IP、端口写错。解决：先运行服务器程序，模拟器使用 10.0.2.2 访问宿主机（真机使用主机局域网 IP），端口与服务器保持一致（8888）。

3. 现象：HTTP 请求返回 404 或连接超时。原因：本机未安装 Tomcat，test.jsp 无法访问。解决：编写 HttpTestServer 模拟 test.jsp 的 GET/POST 处理逻辑并监听 8080 端口；同时把已废弃的 HttpClient 改为 HttpURLConnection。

4. 现象：WebView 打开 HTTP 网页时报 ERR_CLEARTEXT_NOT_PERMITTED。原因：targetSdk 28 以后系统默认禁止明文 HTTP 流量。解决：在 AndroidManifest.xml 的 application 中增加 android:usesCleartextTraffic="true"。

5. 现象：程序运行后定位界面一直没有回调。原因：只声明了权限却没有动态申请，或模拟器没有设置位置。解决：增加 requestPermissions() 动态申请，并在模拟器中用 adb emu geo fix 116.397 39.909 设置经纬度后重新获取。

2 实验收获及体会（本次实验掌握了哪些知识点，学到了什么？可以进行总结在这里，分条写，不要堆在一起）

1. 掌握了 TCP Socket 编程：服务器端用 ServerSocket 监听、accept() 等待连接，客户端用 Socket 连接服务器，双方通过 BufferedReader/PrintWriter 收发数据，理解了流套接字面向连接、可靠传输的特点。

2. 理解了 Android 的线程规范：网络请求必须在子线程执行，结果通过 runOnUiThread()/Handler 回到主线程更新 UI，避免阻塞主线程导致 ANR。

3. 掌握了 HttpURLConnection 发送 GET/POST 请求的完整流程，包括参数拼接与 URL 编码、设置请求头、读取响应状态码和响应流，以及请求异常的处理。

4. 掌握了 WebView 的使用：WebSettings 初始化、WebViewClient 的页面加载回调、loadUrl() 加载网址，并了解了明文流量限制等安全配置。

5. 掌握了 LocationManager 定位：权限声明与运行时申请、LocationListener 回调、requestLocationUpdates() 注册监听，以及模拟器定位的调试方法。

3 不足之处及下一步需改进的地方（实验不足之处，以及需要改进的地方，分条写，不要堆在一起）

1. 程序 1 只实现了单次收发，没有心跳、重连和多客户端会话管理；下一步可以定义消息协议，增加心跳包和断线重连机制，并用线程池管理多个客户端。

2. 程序 2 使用原生 HttpURLConnection，代码较繁琐，也没有缓存和重试；下一步改用 OkHttp/Retrofit，并结合 JSON 解析统一处理请求结果和错误。

3. 程序 3 的 WebView 没有处理 JS 交互、文件下载和页面安全策略；下一步可通过 addJavascriptInterface()、WebChromeClient 增强交互，并限制可访问的域名白名单。

4. 程序 4 只显示单次经纬度，没有逆地理编码和轨迹记录；下一步可接入地图 SDK 显示地图和地址，并针对定位频率、耗电做优化。

5. 界面整体比较简陋，只用了基本的 LinearLayout 和 Button；下一步可使用 RecyclerView、Fragment 或 Compose 优化界面，并补充异常提示和输入校验。